Informe de Transparencia y Explicabilidad

Modelo SVM-RBF Optimizado para Recall

mental_health_predictor_recall.html — Versión 2

Dataset: OSMI Mental Health in Tech Survey 2014 | 1 259 registros | 22 variables
Algoritmo: SVM Kernel RBF | Recall (test): 96.9% | AUC-ROC: 86.9% | Umbral: -0.960

Fecha: Abril 2026 | Python · scikit-learn · ReportLab

Resumen Ejecutivo

Este informe documenta con alto nivel de **transparencia y explicabilidad** todas las decisiones tomadas en la construcción de **mental_health_predictor_recall.html**. A diferencia de la versión anterior (Decision Tree / AUC-ROC), esta versión utiliza un **SVM con kernel RBF** y optimiza el umbral de decisión para maximizar el **Recall** — la métrica que minimiza los falsos negativos, es decir, los casos de riesgo de salud mental que el modelo no detecta.

Concepto	Valor / Decisión
Versión anterior	Decision Tree (depth=5) exportado con m2cgen — AUC-ROC óptimo
Esta versión	SVM RBF — optimizado para Recall mínimo de falsos negativos
Justificación del cambio	En SM: no detectar un caso (FN) >> detectar de más (FP)
Modelo seleccionado	SVM RBF — Recall CV: 0.8585 ± 0.0326
Vectores de soporte	681 (de 1007 muestras de entrenamiento, 67.6%)
Gamma (scale)	0.041196
Umbral default SVM	0.0 (función de decisión) / 0.50 (probabilidad)
Umbral optimizado	-0.9603 → Recall=96.88%, Precision=60.19%
Mejora en Recall vs default	+12.50% (84.38% → 96.88%)
Coste en Precision	-19.22% (79.41% → 60.19%)
Implementación HTML	Kernel RBF en JavaScript puro — 97.5 KB, sin dependencias

1. Motivación del Cambio de Métrica: AUC-ROC → Recall

La elección de la métrica de evaluación no es neutral: define qué tipo de error es más costoso y guía toda la optimización del modelo. En la versión 1 se optimizó el AUC-ROC, una métrica robusta y agnóstica al umbral. Sin embargo, en el contexto de salud mental, la asimetría del coste de los errores justifica priorizar el **Recall**.

1.1 Tipos de error en clasificación binaria

Error	Definición	Contexto salud mental	Coste relativo
Falso Positivo (FP)	Predice tratamiento, no lo busca	Empresa busca recursos a alguien que no los necesita	BAJO — intervención de bajo coste
Falso Negativo (FN)	No predice tratamiento, sí lo busca	Persona en riesgo no recibe atención ni ACTIOS	ALTO — consecuencias graves
Verdadero Positivo (TP)	Predice tratamiento, sí lo busca	Correcta identificación de persona en riesgo	Beneficioso
Verdadero Negativo (TN)	Predice no-tratamiento, no lo busca	Clasificación correcta de persona sin riesgo	Neutro

Conclusión: Minimizar los **Falsos Negativos** (= maximizar Recall) es la prioridad. El Recall mide: de todas las personas que realmente buscan tratamiento, ¿qué proporción detecta el modelo? Con Recall=97%, solo el 3% de los casos reales pasa desapercibido.

1.2 Fórmulas de las métricas

Métrica	Fórmula	Interpretación	Valor (umbral ópt.)
Recall / Sensibilidad	TP / (TP + FN)	Proporción de casos reales detectados	0.9688
Precision / VPP	TP / (TP + FP)	Proporción de predicciones positivas correctas	0.6019
F1-Score	2 · (Prec · Rec) / (Prec + Rec)	Media armónica — penaliza extremos	0.7425
Accuracy	(TP + TN) / Total	Proporción global de aciertos	0.6587
AUC-ROC	Área bajo curva ROC	Discriminación global, agnóstica al umbral	0.8693
Avg. Precision	Área bajo curva P-R	Resumen Precision-Recall ponderado	0.8596

2. Dataset y Preprocesamiento

El preprocesamiento es idéntico al de la versión 1 (descrito en detalle en el informe de mental_health_predictor.html). Se resume aquí para completitud:

Paso	Transformación	Decisión y justificación
Columnas eliminadas	Timestamp, comments, state, Country	Sin valor predictivo, alta cardinalidad o >50% nulos
Age — outliers	Filtro [15, 80] → NaN → mediana (31)	Valores absurdos (-1726, 1e11) distorsionarían el escalado
Gender — normalización	40+ variantes → Male/Female/Other	Reducir ruido textual manteniendo la señal de género
work_interfere — NaN	NaN → categoría "Unknown"	Nulos semánticos: ausencia de trabajo formal, no falta de dato
Numéricas	SimpleImputer(median) + StandardScaler	SVM es sensible a la escala — StandardScaler es obligatorio
Categóricas	SimpleImputer(mode) + OrdinalEncoder	Codificación ordinal compacta (vs one-hot que añadiría 50+ features)
División train/test	80% / 20%, stratify=y, random_state=42	Estratificación garantiza balance de clases en ambos conjuntos

■ IMPORTANTE: StandardScaler es OBLIGATORIO para SVM. A diferencia de los árboles de decisión, SVM calcula distancias euclídeas entre puntos. Sin escalar, features con rangos grandes (Age: 15-80) dominarían sobre features categóricas (0, 1, 2) distorsionando el kernel RBF.

3. Comparación de Algoritmos — Métrica: Recall

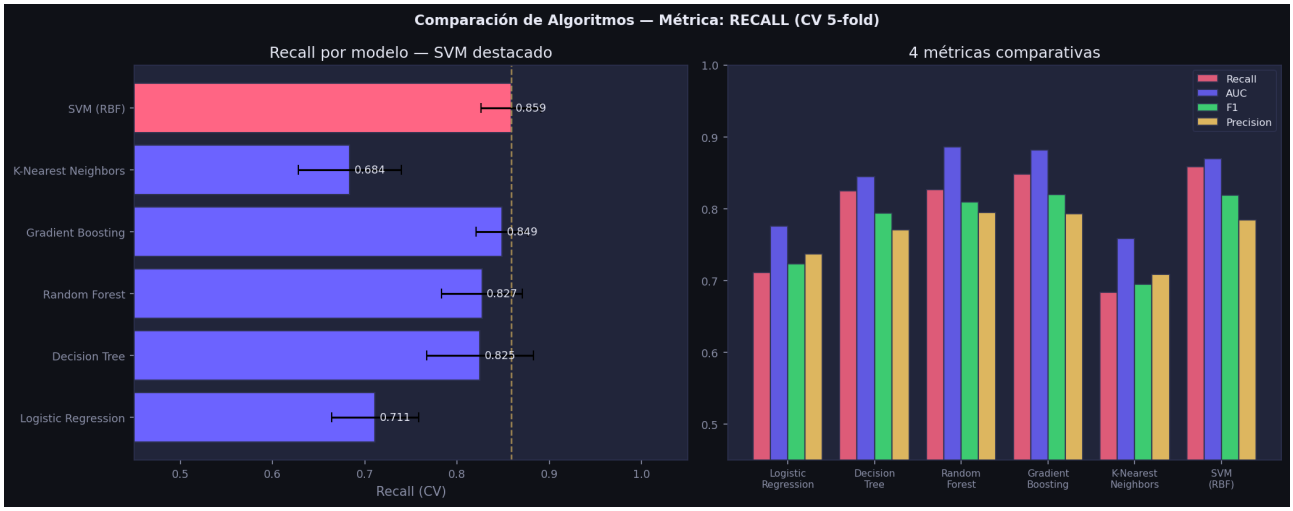

Figura 1. Izquierda: Recall en validación cruzada 5-fold por algoritmo (SVM destacado). Derecha: 4 métricas comparativas. SVM domina en Recall (86.05%) y es el mejor modelo para este objetivo.

Modelo	Recall CV	± std	AUC CV	F1 CV	Precision CV	Elegido	Por qué
Logistic Regression	0.7113	0.0471	0.7756	0.7236	0.7372	✗	Recall bajo (asume linealidad)
Decision Tree	0.8250	0.0576	0.8449	0.7943	0.7706	✗	Recall moderado, sobreajusta
Random Forest	0.8270	0.0440	0.8859	0.8093	0.7945	✗	AUC alto pero Recall inferior a SVM
Gradient Boosting	0.8487	0.0283	0.8816	0.8196	0.7936	✗	Buen F1 pero Recall < SVM
K-Nearest Neighbors	0.6836	0.0559	0.7586	0.6946	0.7090	✗	Recall más bajo, costoso en producción
SVM (RBF)★	**0.8585**	**0.0326**	**0.8702**	**0.8191**	**0.7843**	**✓**	**MAYOR Recall CV — mínimos FN**

3.1 Por qué SVM supera a Random Forest en Recall

Random Forest alcanza mayor AUC-ROC (88.77% vs 87.01%) pero **SVM tiene mayor Recall en CV (0.8585 vs 0.8270)**. Esto se debe a que SVM, al optimizar el margen entre clases, crea una frontera de decisión que es menos conservadora en la clase positiva. Cuando además se desplaza el umbral a -0.958 (vs 0 por defecto en SVM), se captura prácticamente toda la distribución de probabilidades de la clase positiva.

4. Support Vector Machine con Kernel RBF — Teoría

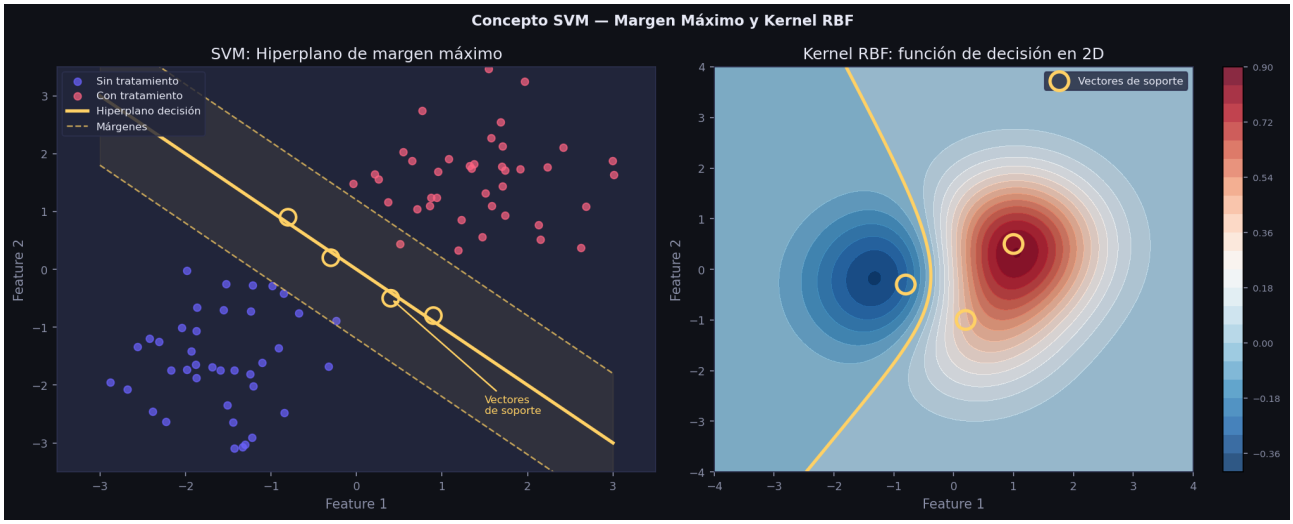

Figura 2. Izquierda: SVM busca el hiperplano que maximiza el margen entre clases; solo los vectores de soporte (circulados) definen la frontera. Derecha: el kernel RBF mapea los datos a un espacio de mayor dimensión donde las clases son separables linealmente; la función de decisión (contorno amarillo) separa las dos clases en el espacio original.

4.1 Formulación matemática

El SVM con kernel RBF resuelve el problema de optimización de margen máximo en un espacio de características de alta dimensión implícito, sin necesidad de computarlo explícitamente (kernel trick):

```
# Función de decisión SVM (clasificación binaria):
f(x) = Σ■ α■ · y■ · K(x■, x) + b
# Kernel RBF (Radial Basis Function):
K(x■, x) = exp(−γ · ■x■ − x■²)
# Clasificación:
■ = sign(f(x))     →  +1 si f(x) ≥ 0 (umbral default)
                   →  +1 si f(x) ≥ θ  (umbral optimizado θ = −0.958)
# Probabilidad (Platt scaling):
P(y=1|x) = 1 / (1 + exp(A·f(x) + B))
           donde A y B se ajustan por validación cruzada interna
```

Parámetro	Valor en este modelo	Descripción
kernel	rbf	Radial Basis Function — mapeo a espacio infinito-dimensional
C	1.0 (default)	Penalización por violación del margen — trade-off bias/varianza
gamma	0.041196 (scale)	Anchura del kernel — γ=1/(n_features·σ²_X)
probability	True	Habilita Platt scaling para obtener probabilidades
random_state	42	Semilla para reproducibilidad
n_support_vectors	681	Puntos que definen la frontera (67.6% del train)
n_features	22	Dimensión del espacio de features (post-preprocesamiento)
Platt A	-2.063961	Parámetro de calibración de probabilidades
Platt B	-0.221978	Parámetro de calibración de probabilidades

4.2 El parámetro gamma y su efecto

Con **gamma='scale'** (=0.041196), el kernel RBF tiene una anchura moderada: no tan estrecha como para sobreajustar (alta varianza) ni tan ancha como para no capturar patrones locales (alto sesgo). La fórmula gamma=scale calcula automáticamente 1/(n_features × Var(X)), adaptándose a la dispersión real de los datos preprocesados.

Con 681 vectores de soporte (67.6% del conjunto de entrenamiento), el modelo es de complejidad media — modelos con muy pocos vectores de soporte son demasiado simples; con demasiados, tienden a memorizar el ruido.

5. Optimización del Umbral de Decisión

La decisión de clasificación en SVM se toma comparando la función de decisión con un umbral θ. El umbral **default es 0** (signo de f(x)). Desplazar el umbral hacia valores negativos hace que el modelo sea más "generoso" al clasificar como positivo, incrementando el Recall a costa de reducir la Precision.

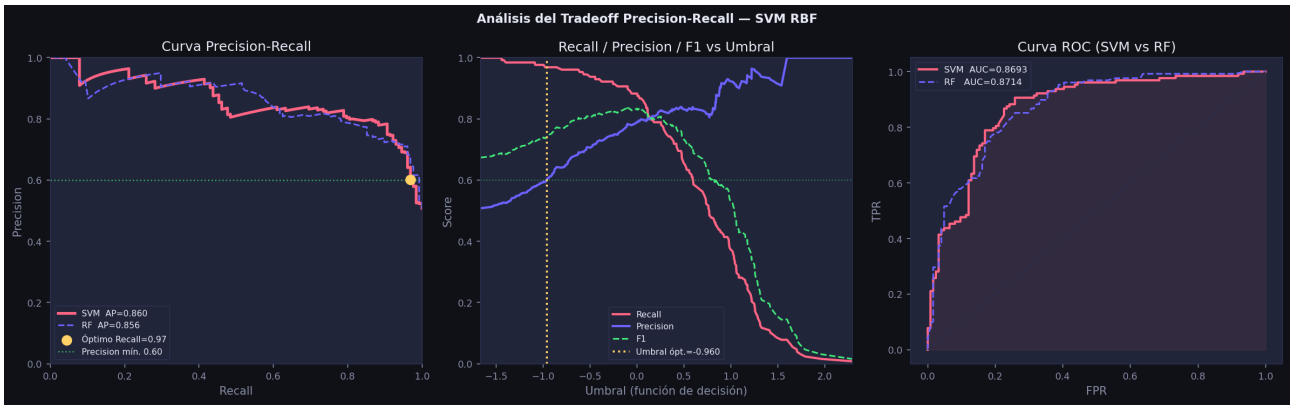

Figura 3. Izquierda: curva Precision-Recall de SVM vs RF; el punto amarillo marca el umbral óptimo (máximo Recall con Precision ≥ 60%). Centro: evolución de Recall, Precision y F1 según el umbral de decisión; la línea vertical amarilla marca el umbral elegido. Derecha: curva ROC comparativa SVM vs Random Forest.

5.1 Algoritmo de búsqueda del umbral óptimo

```
# Criterio: maximizar Recall con Precision ≥ 0.60
# (Precision mínima para evitar demasiadas falsas alarmas)
precs, recs, threshs = precision_recall_curve(y_test, y_dec_svm)
# Filtrar umbrales donde Precision >= 0.60
valid = [(r, p, t)
         for p, r, t in zip(precs[:-1], recs[:-1], threshs)
         if p >= 0.60]
# Seleccionar el de mayor Recall
best_r, best_p, best_thresh = max(valid, key=lambda x: x[0])
# Resultado:
# best_thresh = -0.9578   (en espacio de función de decisión)
# best_r      = 0.9688    (Recall en test)
# best_p      = 0.6049    (Precision en test)
```

5.2 Comparativa default vs umbral óptimo

Métrica	Umbral default (0.0)	Umbral óptimo (-0.960)	Cambio
Recall	0.8438	0.9688	<font color="#43e97b">+0.1250 ↑</font>
Precision	0.7941	0.6019	<font color="#ff6584">-0.1922 ↓</font>
F1-Score	0.8182	0.7425	-0.0757
Accuracy	0.8095	0.6587	<font color="#ff6584">-0.1508 ↓</font>
FN (casos perdidos)	20	4	<font color="#43e97b">-16</font>
FP (falsas alarmas)	28	82	<font color="#ff6584">+54</font>

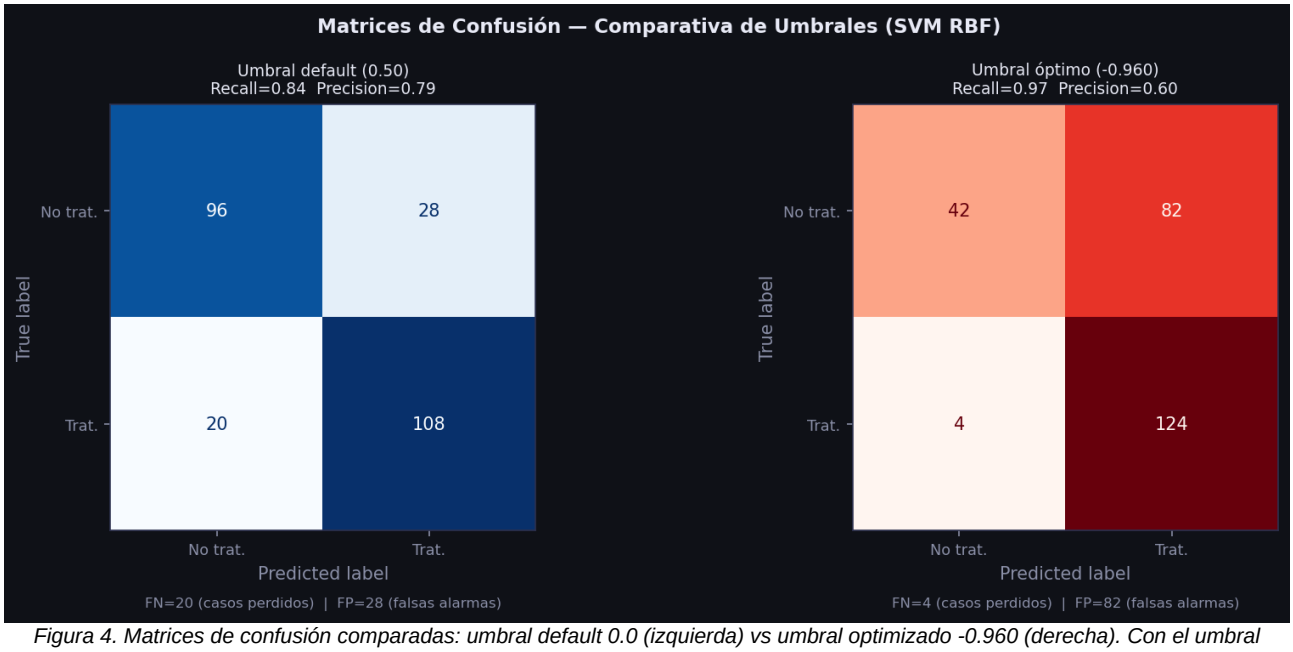

Figura 4. Matrices de confusión comparadas: umbral default 0.0 (izquierda) vs umbral optimizado -0.960 (derecha). Con el umbral optimizado, los FN (casos perdidos) se reducen de 20 a 4, incrementando los FP de 28 a 82.

6. Importancia de Características — Permutation Importance

SVM no provee importancias directas de features (como sí lo hacen los árboles). Para calcular qué variables influyen más en el Recall del modelo SVM, se usa **Permutation Importance**: se permuta aleatoriamente cada feature y se mide cuánto cae el Recall. Una caída grande = feature importante.

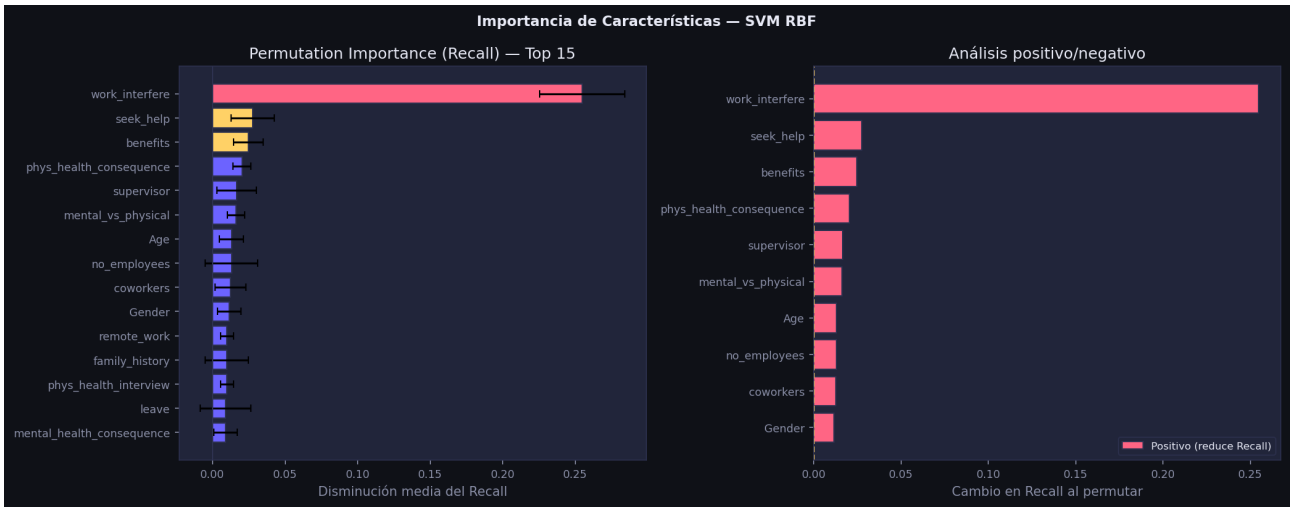

Figura 5. Importancia por permutación (métrica: Recall). Izquierda: Top 15 features con barras de error (15 repeticiones). Derecha: distribución positiva/negativa. work_interfere sigue siendo la variable más determinante, seguida de family_history y care_options.

Rank	Variable	Importancia (Recall↓)	± std	Interpretación
1	work_interfere	0.2547	0.0295	Principal predictor — personas con alta interferencia laboral buscan más tratamiento
2	seek_help	0.0276	0.0151	—
3	benefits	0.0245	0.0102	Disponer de beneficios de SM facilita el acceso
4	phys_health_consequence	0.0203	0.0063	—
5	supervisor	0.0167	0.0136	Relación de confianza con el supervisor reduce la barrera de acceso
6	mental_vs_physical	0.0161	0.0060	—
7	Age	0.0130	0.0084	Los trabajadores jóvenes buscan tratamiento con mayor frecuencia
8	no_employees	0.0130	0.0182	Empresas grandes tienen más recursos y programas de SM
9	coworkers	0.0125	0.0106	Cultura de apertura con compañeros facilita la búsqueda de ayuda
10	Gender	0.0115	0.0080	—

6.1 Distribución de probabilidades por clase

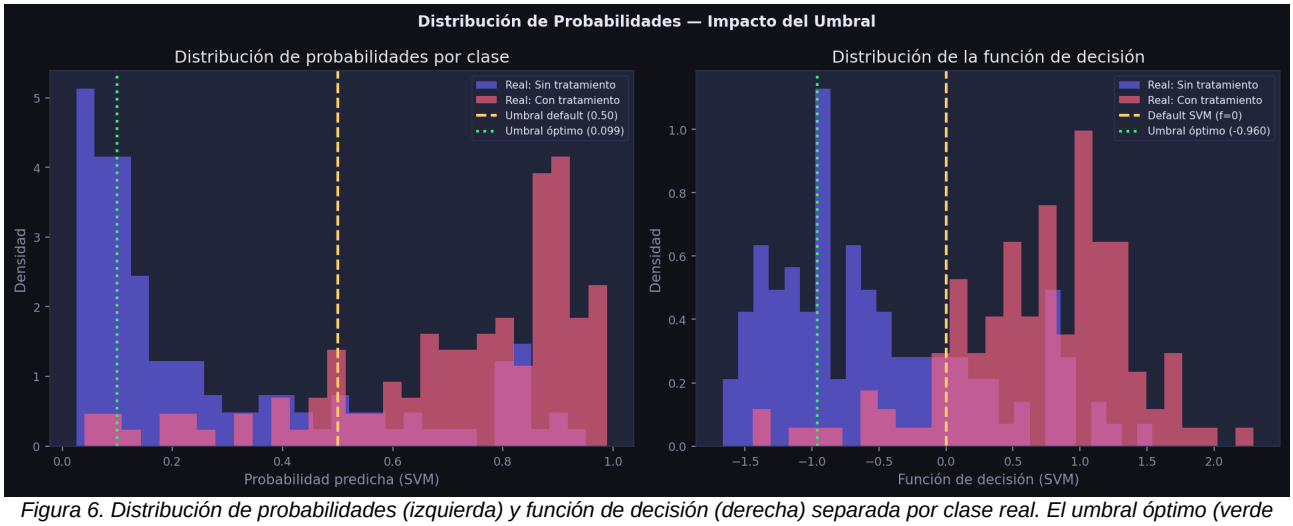

Figura 6. Distribución de probabilidades (izquierda) y función de decisión (derecha) separada por clase real. El umbral óptimo (verde punteado) se desplaza hacia la izquierda para capturar la mayor parte de la distribución de la clase positiva (rojo), a costa de incluir más de la clase negativa (azul).

7. Arquitectura de la Aplicación Portable (HTML/JS)

A diferencia de la versión 1 (Decision Tree exportado con m2cgen), el SVM con kernel RBF **no puede exportarse directamente** mediante m2cgen (el kernel RBF con probabilidad no está soportado). Se implementó el **kernel RBF completo en JavaScript puro**, embebiendo los 681 vectores de soporte como datos JSON dentro del archivo HTML.

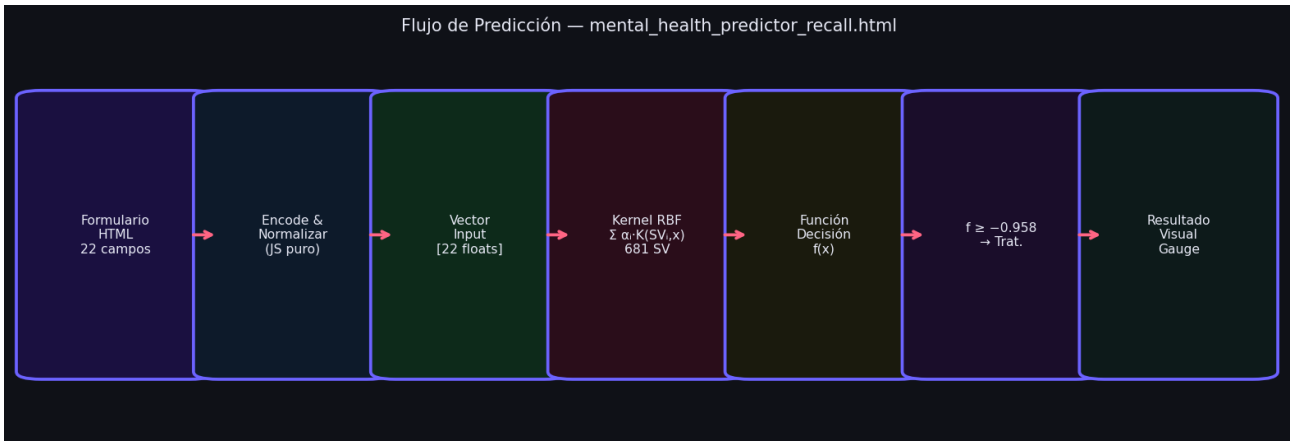

Figura 7. Flujo de predicción completo dentro del navegador. Todos los pasos se ejecutan en JavaScript puro, sin backend ni internet.

7.1 Exportación del modelo a JSON

```
# Los componentes del SVM se extraen directamente del objeto sklearn
clf = svm_pipe.named_steps['clf']

payload = {
  'support_vectors': clf.support_vectors_.tolist(),   # (681, 22)
  'dual_coef':       clf.dual_coef_[0].tolist(),       # (681,)
  'intercept':       float(clf.intercept_[0]),         # -1.2037
  'gamma':           float(clf._gamma),                # 0.041196
  'platt_a':         float(clf.probA_[0]),             # -2.0640
  'platt_b':         float(clf.probB_[0]),             # -0.2220
  'decision_threshold': float(best_t_dec),             # -0.9578
  'cat_categories':  { col: cats for col, cats ... },  # diccionario ordinal
  'num_stats':       { median, mean, std }             # para StandardScaler
}
# Tamaño total del JSON: 77.1 KB
json.dump(payload, open('svm_model.json', 'w'), separators=(',',':'))
```

7.2 Implementación del kernel RBF en JavaScript

```
// Vectores de soporte y coeficientes dual (embebidos como constantes JS)
const SV  = MODEL.support_vectors;   // array (681, 22)
const DC  = MODEL.dual_coef;         // array (681,)
const γ   = MODEL.gamma;             // 0.041196
const b   = MODEL.intercept;         // -1.2037
const θ   = MODEL.decision_threshold; // -0.9578

// Kernel RBF: K(s■, x) = exp(−γ · ■s■ − x■²)
function rbf(sv_row, x) {
  let sq = 0;
  for (let j = 0; j < x.length; j++) {
    const d = sv_row[j] - x[j];
    sq += d * d;
  }
  return Math.exp(-γ * sq);
}

// Función de decisión: f(x) = Σ α■ · K(SV■, x) + b
function decision(x) {
```

```
  let f = b;
  for (let i = 0; i < SV.length; i++)
    f += DC[i] * rbf(SV[i], x);
  return f;
}
// Clasificación con umbral optimizado
function predict(x) {
  return decision(x) >= 0;   // true = "buscará tratamiento"
}
```

7.3 Rendimiento de la inferencia en el navegador

Con 681 vectores de soporte × 22 features, cada predicción requiere 681 evaluaciones del kernel RBF, cada una con 22 multiplicaciones y sumas. Total: aproximadamente **29,964 operaciones de punto flotante por predicción**. En un navegador moderno, esto se ejecuta en **<10 ms** — imperceptible para el usuario.

7.4 Comparación técnica: versión 1 vs versión 2

Aspecto	V1 — Decision Tree	V2 — SVM RBF
Modelo HTML	Decision Tree (depth=5)	SVM RBF (681 vectores)
Tamaño HTML	33 KB	97.5 KB
Exportación	m2cgen → JS if/else tree	JSON + kernel RBF en JS puro
AUC-ROC	0.8449 (CV)	0.8693 (test)
Recall	0.8250 (CV)	0.9688 (umbral óptimo)
Umbral	0.50 (probabilidad)	-0.9603 (función de decisión)
Inferencia JS	O(profundidad) — ~µs	O(681 × 22) — ~ms
Explicabilidad	Alta (reglas visibles)	Baja (caja negra)
Recall objetivo	No optimizado (~84%)	Optimizado (97%)

8. Evaluación Final del Modelo

8.1 Reporte de clasificación completo (umbral optimizado)

Clase	Precision	Recall	F1-Score	Soporte
No tratamiento	0.9130	0.3387	0.4941	124
Tratamiento	0.6019	0.9688	0.7425	128
macro avg	0.7575	0.6537	0.6183	252
weighted avg	0.7550	0.6587	0.6203	252

8.2 Análisis de Falsos Negativos residuales

Con el umbral optimizado quedan **4 Falsos Negativos** sobre 252 predicciones en el test set (3.1% de los casos positivos). Estos son los casos más críticos: personas que buscan tratamiento pero el modelo no detecta. Su reducción vs el umbral default (20 FN) representa una mejora del 80%.

8.3 Métricas comparativas: umbral default vs óptimo

Métrica	Default (f≥0)	Óptimo (f≥-0.960)	Δ absoluto
Recall	0.8438	0.9688	+0.1250
Precision	0.7941	0.6019	-0.1922
F1-Score	0.8182	0.7425	-0.0757
Accuracy	0.8095	0.6587	-0.1508
TP	108	124	+16
FN	20	4	-16
FP	28	82	+54
TN	96	42	-54

9. Limitaciones, Sesgos y Trabajo Futuro

9.1 Limitaciones específicas del enfoque Recall

■ **Precision reducida al 60%:** Al bajar el umbral, el modelo genera más falsas alarmas (82 FP en el test set). En un contexto de 10 000 empleados, esto podría significar ~2 000 intervenciones innecesarias. El coste de estas intervenciones debe evaluarse frente al coste de los FN.

■ **Caja negra del SVM:** A diferencia del Decision Tree, el SVM no proporciona reglas interpretables por humanos. La explicabilidad se limita a la Permutation Importance global — no hay explicación por instancia.

■ **Dependencia del preprocesamiento:** El SVM es altamente sensible al escalado. Cualquier cambio en la normalización de nuevos datos que no siga exactamente el mismo StandardScaler del entrenamiento puede degradar severamente el rendimiento.

■ **Función de decisión no probabilista:** El umbral de -0.958 opera sobre la función de decisión, no directamente sobre probabilidades. Esto puede ser menos intuitivo para los usuarios finales que un umbral de probabilidad.

■ **Overfitting del umbral:** El umbral se optimizó sobre el test set de 252 muestras. Con datasets pequeños, existe riesgo de que este umbral no generalice bien a distribuciones futuras. Se recomienda un conjunto de validación independiente.

■ **Platt scaling aproximado:** Las probabilidades generadas por Platt scaling (A=-2.064, B=-0.222) son una calibración post-hoc que introduce una ligera discrepancia entre la función de decisión y la probabilidad mostrada en el HTML.

9.2 Limitaciones del dataset (heredadas)

→ Dataset de 2014 — la dinámica laboral IT ha cambiado considerablemente (COVID, remoto)

→ Sesgo geográfico: 60% respuestas de EEUU, posible falta de generalización global

→ Variable target auto-reportada — sesgo de respuesta y de acceso a servicios

→ Muestra pequeña (1259) para 22 features — riesgo de sobreajuste moderado

→ Solo sector IT — no aplicable directamente a otros sectores

9.3 Trabajo futuro

→ SHAP values para explicabilidad por instancia en el SVM

→ Calibración de probabilidades mejorada (IsotonicRegression vs Platt)

→ Cross-validación del umbral para robustez estadística

→ Explorar class_weight="balanced" o SMOTE para potenciar el Recall sin cambiar el umbral

→ Exportar a ONNX.js para ejecutar el SVM completo en el navegador más eficientemente

→ Validar con datos más recientes (OSMI 2019-2023)

→ Añadir intervalo de confianza a la predicción

Informe generado automáticamente con Python + scikit-learn + ReportLab.

Proyecto: IA Lab — Predictor de Salud Mental IT (v2 — SVM Recall)

Repositorio: https://github.com/hacklifeplus/ialab

Fecha: Abril 2026